



Dictionary methods

Previously, you were introduced to dictionaries and learned a little bit about how they work. Let's continue our exploration of dictionaries and how to use them. Let's consider a previous example and revisit the women's basketball team roster. Recall that the roster was encoded as a list of tuples. Each tuple represented the name, age, and position of a player on the team. The list of tuples was useful when we had a single team and one player per position. What if we add more players beyond the starting five? A dictionary can help us organize the data according to our specific needs. For example, what if we want to be able to look up players by their position? We can create a dictionary where the positions are the keys and the players are the values, each represented as a tuple that contains two values: their name and age. We could retype the information into a dictionary or cut and paste it, but if you find yourself doing these things, you can take this as an opportunity to improve your coding skills. Consider that this is the information for just 10 players; what if we had data for the whole league? We can convert this information to a dictionary with a for loop and some conditional logic. We'll begin by instantiating an empty dictionary and designing it to a variable named "new_team."

The idea is to loop over each tuple in the list, extract the position and assign it as a dictionary key, and extract the player's name and age and assign them as a tuple within a list, representing the value for that key. The process would repeat for each iteration of the loop, until all of the players are recorded in the dictionary. Notice that each position is only represented once as a key. So the final dictionary has five keys, and there are two players in the list at each key's value. Now let's write the loop. First, we'll assign the empty dictionary to a variable called "new_team." Then we'll write a for loop that unpacks the information in the original tuples. "For name, age, position in team."

And here's where the conditional logic comes in. If the position already exists as a keyword in our dictionary, then we want to append the name and age tuple to the list of tuples. Remember, the value for each key will be a list that contains tuples of player information. If the position does not already exist as a keyword in the dictionary, we'll have to assign it. We'll use an else statement to do this. With only a few lines of code, we have converted our list of tuples to a dictionary. Let's check that it works.

It sure does. Creating dictionaries this way is a common practice in data analytics with Python, so learning this process will help make you a more capable data professional. Now, let's learn about some useful methods that we can use on dictionaries to really take advantage of their power. Specifically, we'll discuss the keys, values, and items methods. If you run a loop over a dictionary, the loop will only access the keys, not the values. For example, if we loop over the dictionary we just created and print each iteration, the computer will return five positions, the keys of the dictionary. But you don't need to write a loop every time you want to access the keys of your dictionary.

That's what the keys method is for. The keys method lets you retrieve only the dictionary's keys. Returning to our "new_team" dictionary, when we apply the keys method to the dictionary, the computer returns a list of all its keys. Similarly, the values method lets you retrieve only the dictionary's values. When applied to our "new_team" dictionary, we get the values returned as a list. Since our values are lists of tuples, it means the result of calling this method is a list of lists of tuples. But what if you want to access both the keys and their values?

You can, using the items method, which lets you retrieve both the dictionary's keys and values. Let's use a loop to print what the items method returns so the output is prettier.

Dictionaries make storing and retrieving data fast and efficient. Keep exploring the many things you can do with them. With time, you'll find that they become an important tool in your data analytics toolbox.

keys()

A dictionary method to retrieve only the dictionary's keys

values()

A dictionary method to retrieve only the dictionary's values

items()

A dictionary method to retrieve both the dictionary's keys and values